Test Time & Vector Memory Optimization Agent

Project Technical Architecture & Answers to Core Evaluation Questions

1. What is the agent responsible for?

The **Test Time Optimization Agent** is responsible for solving the semiconductor Automated Test Equipment (ATE) vector memory and test time bottleneck in the testing path: *STIL → Vector Memory (RAM) → DUT Pins*.

- **Pre-Process (Vector Memory & Test Time Optimization):** Ingests large scan pattern suites in Standard Test Interface Language (STIL), compresses the multi-channel scan bitstreams using an LSTM neural network, and autonomously selects an optimal, high-diversity subset of patterns (**'High Recommended Patterns'**) while pruning redundant ones (**'Low Risk Patterns'**). This saves vector RAM and playback time, preventing ATE testers from pausing for expensive memory reload cycles.
- **Post-Process (Silicon Failure Verification):** Ingests real ATE tester output logs directly in the client, parses per-pattern hardware pass/fail statuses, and cross-references actual silicon defect detections against kept vs. skipped patterns to empirically prove that defect coverage is preserved with zero quality loss.

2. Which AI/ML model is being used?

The only AI/ML model used in the application is the **LSTM (Long Short-Term Memory) Neural Network** (implemented as PatternLSTMCompressor in PyTorch).

- **Architecture:** A 2-layer recurrent LSTM sequence model with an input linear projection layer (with Tanh non-linear activation), optional neural dropout regularization, and an output bottleneck linear projection layer.
- **Operation:** It takes multi-channel scan bitstreams (batch, sequence_length, channels) clock cycle by clock cycle and compresses each complete scan pattern into a compact, fixed **32-dimensional continuous embedding vector** that preserves the spatial and temporal stimulus of the entire scan chain.

3. Which algorithms are being used?

The core algorithm is the **LSTM (Long Short-Term Memory) Sequence Compression Algorithm** (paired with greedy farthest-point diversity selection). Instead of performing computationally intractable bit-by-bit comparisons across millions of clock cycles, the LSTM compresses the scan bitstream through the following official algorithmic stages:

- **1. Input Linear Projection Layer (input_proj):** Reads raw 0/1 binary scan bits across all chip channels at each clock cycle and maps them into an internal hidden feature space using a Hyperbolic Tangent (&tanh;) activation function.
- **2. LSTM Recurrent Gating Mechanism:** Sequentially processes clock cycles and maintains long-term memory across long scan chains using four internal gating equations:
  - **Forget Gate (f_t):** Filters out repetitive, uninformative bit sequences.
  - **Input Gate (i_t):** Identifies and captures novel, critical bit transitions in the active cycle.
  - **Cell State Update (C_t):** Acts as the continuous memory conveyor belt, preserving cumulative scan context across hundreds of cycles.
  - **Output Gate (o_t):** Modulates the updated cell state to produce the cycle's **Hidden State (h_t)**.
- **3. Final Hidden State Extraction (last_hidden_state):** Once the entire clock cycle sequence of a pattern is read, the algorithm extracts the last hidden state of the top LSTM layer, encapsulating the cumulative structural stimulus of the pattern in a single vector.
- **4. Output Linear Bottleneck Projection Layer (out_proj):** Compresses the internal hidden state down to a compact, fixed 32-dimensional continuous embedding vector.
- **5. L2 Vector Normalization (normalize):** Applies Euclidean L2 normalization ($\hat{e} = e / \|e\|_2$) to project all pattern embeddings onto a unit hypersphere, allowing instant geometric distance measurement.
- **6. Greedy Farthest-Point Diversity Algorithm (diversity_ranking):** Seeds with initial pattern P_0 and iteratively chooses the pattern with the maximum Euclidean distance to all previously selected patterns, maximizing structural test diversity across scan chains.

4. What data sources does the agent use?

The agent operates on two primary semiconductor test data sources:

1. **STIL Files (.stil):** Standard Test Interface Language files generated by commercial ATPG EDA tools (Synopsys TetraMAX, Siemens Tessent). They contain DUT pin declarations, scan chain lengths, cycle timing parameters, and multi-channel 0/1 scan vectors.
2. **ATE Tester Execution / Fail Logs:** Hardware test logs produced by ATE testers (in .log, .txt, .csv formats). These record pattern execution headers (e.g., P12 | CH3 ...), expected vs. actual pin states, and hardware channel statuses (STATUS:F or STATUS:P).

5. What algorithm is used for PASS/FAIL classification?

Deterministic ATE Pattern Comparator & Regular Expression Parser (No ML model is used for this).

PASS/FAIL classification is performed deterministically by reading tester hardware logs directly:
• If an ATE tester channel encounters an unexpected bit output during playback, it flags STATUS:F → classified as a **FAIL**.
• If all pin outputs match expected vector responses across all channels, it records STATUS:P → classified as a **PASS**.
• The agent uses a regex parser to extract pattern IDs and channel statuses, automatically maps 1-based tester log labels (P1..P1000) to 0-based STIL IDs (P0..P999), and computes aggregate fail/pass metrics across dies.

6. Which test parameters are most important?

The key parameters that govern optimization and hardware execution include:

Parameter	Code Variable	Engineering Importance
Peak Vector Memory	peak_vector_mb	Primary tester RAM constraint; determines if pattern suite fits resident in memory or triggers costly multi-load pauses.
Total Cycle Count	n_cycles / total_cycles	Total clock cycles across scan chains; directly dictates tester playback time on the DUT pins.
Tester Clock Period	period_ns	Nanoseconds per cycle; defines the test frequency at which vectors are driven into the device.
DUT Pins & Channels	n_pins, n_channels	Number of tester channels driven simultaneously; determines vector memory byte width per cycle.
Scan Chain Bits (0/1)	data / bits	Raw structural stimulus across scan chains; bit differences drive diversity and fault coverage.
Pattern Fail Count	fails, passes	Frequency of silicon defect detection recorded in ATE tester logs.
Marginal Diversity Gain	gain, stop_ratio	The incremental embedding distance threshold used to determine pattern diversity.

7. Summary about your application

The **Test Time Optimization Agent** is an end-to-end platform designed to solve the semiconductor Automated Test Equipment (ATE) vector memory bottleneck. It integrates a live web dashboard, a streaming server, and a PyTorch backend engine.

1. The Vector RAM Bottleneck: In semiconductor manufacturing, ATPG tools generate massive scan pattern suites in STIL format. When compiled into tester memory, these patterns often exceed the physical onboard RAM of ATE testers. As a result, the tester must repeatedly pause, flush memory, and reload new vectors over slow buses, dramatically increasing test time per chip and driving up test costs.

2. STIL Streaming & ATE Modeling: The agent streams large STIL files pattern-by-pattern without overwhelming system RAM. It extracts scan-in channels across chip pins, expands run-length compressed bits, and accurately models tester memory consumption and test playback time based on pin count and clock cycles.

3. LSTM Sequence Compression & Diversity Selection: Each pattern's scan bitstream is processed by a **2-layer PyTorch LSTM** that compresses thousands of bits into a compact **32-dimensional embedding vector**. Using greedy farthest-point diversity ranking, the agent identifies and retains an optimal subset of structurally distinct patterns (**'High Recommended Patterns'**) while safely skipping redundant ones (**'Low Risk Patterns'**), eliminating memory reload delays.

4. Silicon Log Validation: In the post-process phase, the agent ingests real ATE tester logs directly in the browser. It cross-references actual silicon failures to confirm that defect-detecting patterns are preserved in the kept set, and provides interactive bit-level comparisons highlighting differences in yellow.

Evaluation Parameter	Baseline (Without Agent)	Optimized (With Agent)	Net Benefit & ROI

STIL Patterns Loaded	100% of ATPG patterns (P0..P999)	Optimized diverse subset	**Substantial reduction** in test volume
Vector Memory RAM	Exceeds tester RAM capacity	Fits resident in single load	**Zero tester reload pauses**
Test Playback Time	Full suite duration + reload delays	Continuous single-load execution	**Significant test time savings**
Tester Utilization	Stalled during bus memory reloads	Continuous run at full clock speed	**Maximum ATE efficiency**
Silicon Defect Coverage	Standard baseline coverage	Preserved via LSTM diversity	**Zero quality loss / zero escapes**
Silicon Validation	Manual review of tester logs	Automated log parsing & bit diffs	**Instant silicon verification**
Manufacturing Cost	High test cost per wafer touch	Minimized test cell time	**Direct operational cost savings**